

# CCA-F\_60\_Mock\_MCQs

## CCA-F Mock Exam — 60 Questions

### Claude Certified Architect - Foundations (CCAR-F)

**Time limit: 120 minutes · Pass mark target: 43/60 (~72%)**

Questions are weighted to the official blueprint and grouped by the six official exam scenarios, exactly as the real exam presents them.

| Domain                                          | Questions |
|-------------------------------------------------|-----------|
| 1. Agentic Architecture & Orchestration (27%)   | Q1-Q16    |
| 2. Tool Design & MCP Integration (18%)          | Q17-Q27   |
| 3. Claude Code Configuration & Workflows (20%)  | Q28-Q39   |
| 4. Prompt Engineering & Structured Output (20%) | Q40-Q51   |
| 5. Context Management & Reliability (15%)       | Q52-Q60   |

**Answers and justifications begin on the "ANSWER KEY" section below. Do the whole paper first.**

## PART 1 — QUESTIONS

### Domain 1 — Agentic Architecture & Orchestration

**Q1.** *(Scenario: Customer Support Resolution Agent)* Your agentic loop terminates early, sometimes before the refund is actually processed. Reviewing the code, you find the loop exits when the assistant response contains any text content. What is the correct loop termination condition?

- A. Terminate when the response contains no `tool_use` blocks in its content array  
B. Terminate when `stop_reason` equals `"end_turn"`; continue when it equals `"tool_use"`  
C. Terminate after a maximum of 10 iterations to bound cost  
D. Terminate when the model's text output includes a completion phrase such as "I've completed"

**Q2.** After executing a tool the agent requested, what must your loop do before the next API call so Claude can reason about what it learned?

- A. Append the tool result to the conversation history as a message  
B. Store the result in an external vector database for retrieval  
C. Replace the previous assistant message with the tool result  
D. Include the result only in the system prompt to save tokens

**Q3.** Which statement best distinguishes a genuinely agentic system from a workflow?

A. Agentic systems use more tools than workflows B. Agentic systems run asynchronously; workflows run synchronously C. In an agentic system Claude reasons about which tool to call next based on current context, rather than following a pre-configured decision tree or fixed tool sequence D. Agentic systems require MCP; workflows use direct function calls

---

**Q4.** (*Scenario: Multi-Agent Research System*) Your coordinator agent is configured but never spawns any subagents — it attempts to do all research itself. Its `AgentDefinition` for each subagent looks correct. What should you check first?

A. Whether the subagents have their own `allowedTools` configured B. Whether the coordinator's `allowedTools` includes `"Task"` C. Whether `fork_session` is enabled on the coordinator D. Whether the subagents' system prompts mention the coordinator

---

**Q5.** The research pipeline currently takes 4 minutes because the web search subagent, document analysis subagent, and citation subagent run one after another, even though they are independent. How do you make them run in parallel?

A. Set `tool_choice: "any"` on the coordinator B. Have the coordinator emit multiple `Task` tool calls within a single response C. Configure each subagent with `context: fork` in its definition D. Increase `max_tokens` so the coordinator can issue more calls per turn

---

**Q6.** The synthesis subagent produces a report that contradicts the web search subagent's findings and omits several key facts. Investigation shows the synthesis subagent was invoked with only the instruction "synthesise the research findings." What is the most likely cause?

A. The synthesis subagent's model tier is too small for the task B. Subagents do not automatically inherit the coordinator's conversation history, so the prior findings were never provided in the synthesis prompt C. The coordinator called the synthesis subagent before search completed D. The synthesis subagent needs `allowedTools` to include `"Task"`

---

**Q7.** Your final reports on "impact of AI on creative industries" consistently cover only visual arts. Every subagent completes successfully and correctly. The coordinator logs show subtasks: "AI in digital art creation," "AI in graphic design," "AI in photography." What is the root cause?

A. The document analysis agent filters out non-visual sources through overly restrictive relevance criteria B. The web search agent's queries need broadening across more sectors C. The coordinator's task decomposition is too narrow, so subagent assignments don't cover the topic's relevant domains D. The synthesis agent lacks instructions to identify coverage gaps

---

**Q8.** (Scenario: Customer Support Resolution Agent) In 12% of cases the agent skips `get_customer` and calls `lookup_order` using only the customer's stated name, occasionally causing refunds on misidentified accounts. What most effectively fixes this?

- A. Add few-shot examples showing the agent always calling `get_customer` first
- B. Strengthen the system prompt to state that verification is mandatory before order operations
- C. Add a programmatic prerequisite that blocks `lookup_order` and `process_refund` until `get_customer` returns a verified customer ID
- D. Deploy a routing classifier that enables only the appropriate subset of tools per request type

---

**Q9.** Company policy caps agent-issued refunds at \$500; above that a human must approve. Which implementation gives a guaranteed, deterministic compliance outcome?

- A. A system prompt rule stating refunds above \$500 must be escalated
- B. Few-shot examples demonstrating escalation for high-value refunds
- C. A tool call interception hook that blocks `process_refund` calls above \$500 and redirects to the human escalation workflow
- D. A `PostToolUse` hook that logs refunds above \$500 for later audit

---

**Q10.** Your MCP tools return dates in three formats: Unix timestamps from the orders service, ISO-8601 from the billing service, and numeric status codes from the shipping service. The agent frequently misinterprets them. What is the cleanest fix?

- A. Instruct the model in the system prompt to normalise all date formats itself
- B. Implement a `PostToolUse` hook that normalises tool results into a consistent format before the model processes them
- C. Add few-shot examples showing each format being interpreted correctly
- D. Force `tool_choice` to a `normalize_data` tool after every call

---

**Q11.** A customer message contains three separate issues: a damaged item, an incorrect billing charge, and an address change. What is the recommended handling pattern?

- A. Handle only the first issue and ask the customer to open separate tickets for the others
- B. Escalate immediately to a human because multi-concern requests exceed agent scope
- C. Decompose into distinct items, investigate each in parallel using shared context, then synthesise a single unified resolution
- D. Process them strictly sequentially, closing the conversation after each

---

**Q12.** The agent escalates a billing dispute mid-conversation. The receiving human agent has no access to the conversation transcript. What must the handoff include?

- A. A link to the conversation ID so the human can request the transcript later
- B. A structured summary containing customer ID, root cause analysis, refund amount, and recommended action
- C. The full

raw message history serialised as JSON D. A confidence score and sentiment rating for the conversation

---

**Q13.** (*Scenario: Developer Productivity*) You are asked to "add comprehensive tests to a legacy codebase" with unknown structure and undocumented dependencies. Which decomposition strategy fits best?

- A. A fixed prompt chain: generate tests file by file in alphabetical order
  - B. Dynamic adaptive decomposition: map structure first, identify high-impact areas, then build a prioritised plan that adapts as dependencies are discovered
  - C. A single prompt containing the entire codebase and a request for full coverage
  - D. Batch API submission of every file with a shared test-generation prompt
- 

**Q14.** For a predictable multi-aspect code review where the required checks are known in advance, which pattern is appropriate?

- A. Dynamic decomposition driven by intermediate findings
  - B. Prompt chaining — sequential focused passes, e.g. per-file analysis then a cross-file integration pass
  - C. A single monolithic review prompt covering all concerns at once
  - D. Parallel independent agents each given the whole PR with no coordination
- 

**Q15.** You are comparing two refactoring strategies and want both explorations to start from the same completed codebase analysis, without redoing it. Which feature fits?

- A. `--resume <session-name>`
  - B. `/compact`
  - C. `fork_session`
  - D. `context: fork` in a SKILL.md
- 

**Q16.** You return to an investigation session after a teammate refactored several files you had previously analysed. The prior tool results no longer reflect the code. What is the more reliable approach?

- A. Resume the session unchanged; the model will re-read files as needed
  - B. Resume the session and increase `max_tokens`
  - C. Start a new session with a structured summary of the still-valid findings, rather than resuming into stale tool results
  - D. Resume and run `/compact` to remove the stale results
- 

## Domain 2 — Tool Design & MCP Integration

---

**Q17.** Production logs show the agent calls `get_customer` when users ask "check my order #12345", instead of `lookup_order`. Both tools have minimal descriptions ("Retrieves customer information" / "Retrieves order details") and accept similar identifier formats. What is the most effective **first step**?

- A. Add 5-8 few-shot examples to the system prompt routing order queries to `lookup_order`
- B. Expand each tool's description to include input formats, example queries, edge cases, and boundaries explaining when to use it versus similar tools
- C. Implement a routing layer that

parses input and pre-selects the tool by keyword and identifier pattern  
D. Consolidate both into a single `lookup_entity` tool that determines the backend internally

---

**Q18.** A research agent has two tools, `analyze_content` and `analyze_document`, with near-identical descriptions, causing constant misrouting. Which change best eliminates the overlap?

A. Delete `analyze_content` and route everything through `analyze_document`  
B. Rename `analyze_content` to `extract_web_results` and rewrite its description to be web-specific  
C. Add a `source_type` parameter to both tools  
D. Set `tool_choice: "any"` so the model must pick one

---

**Q19.** Even after excellent tool descriptions, the agent still misroutes certain requests. What else should you review?

A. The `max_tokens` setting  
B. The MCP server's transport protocol  
C. The system prompt, for keyword-sensitive instructions that create unintended tool associations and override the descriptions  
D. The order of tools in the tools array

---

**Q20.** Your `lookup_order` tool returns `{"isError": true, "message": "Operation failed"}` for every failure type. The agent retries invalid-input errors endlessly and gives up on transient timeouts. What should you return instead?

A. A plain-text error string the agent can read naturally  
B. `isError: true` plus structured metadata including `errorCategory` (transient/validation/permission), an `isRetryable` boolean, and a human-readable description  
C. A HTTP status code in the message field  
D. `isError: false` with an empty result set so the agent continues gracefully

---

**Q21.** A refund is denied because the return window has expired. How should the MCP tool report this?

A. `errorCategory: "transient"` with `isRetryable: true`  
B. A business error with `retryable: false` and a customer-friendly explanation the agent can relay  
C. A successful empty response so the agent moves on  
D. A permission error, since the agent lacks authority to override policy

---

**Q22.** The web search subagent's query times out. Separately, a different query succeeds but returns zero matching documents. Why must these be reported differently?

A. Timeouts are billed differently from successful calls  
B. They are semantically different: one is an access failure that may warrant a retry or alternative approach, the other is a valid result meaning "nothing matches" — conflating them leads to wrong recovery decisions  
C. Empty results must always trigger escalation to a human  
D. Only timeouts should set the `isError` flag; both should be retried

---

**Q23.** A synthesis agent has been given all 18 tools available in the system "so it never gets blocked." It now performs poorly, sometimes attempting web searches instead of synthesising. What principle was violated and what is the fix?

- A. Context window limits; reduce `max_tokens` B. Least privilege / scoped tool access; restrict the agent to tools relevant to its role, adding narrow cross-role tools only for high-frequency needs C. Prompt caching; enable caching for the tool definitions D. Determinism; set `tool_choice` to forced selection
- 

**Q24.** The synthesis agent needs to verify facts. Evaluation shows 85% are simple lookups (dates, names, statistics) and 15% need deeper investigation. Round trips through the coordinator add 40% latency. Best approach?

- A. Give the synthesis agent access to all web search tools so it never round-trips B. Give it a scoped `verify_fact` tool for simple lookups, keeping complex verifications routed through the coordinator C. Have it batch all verification needs and return them to the coordinator at the end of its pass D. Have the web search agent proactively cache extra context around every source
- 

**Q25.** You need to guarantee that `extract_metadata` runs before any enrichment tools. Which configuration achieves this?

- A. `tool_choice: "auto"` with an instruction in the system prompt B. `tool_choice: "any"` C. `tool_choice: {"type": "tool", "name": "extract_metadata"}` on the first turn, then process later steps in follow-up turns D. Listing `extract_metadata` first in the tools array
- 

**Q26.** Your team shares a GitHub MCP server across the project, and you also run a personal experimental server. The GitHub server needs a token that must not be committed. How do you configure this?

- A. Both servers in `.mcp.json`, with the token hard-coded and the file gitignored B. GitHub server in project-scoped `.mcp.json` using `${GITHUB_TOKEN}` environment variable expansion; personal server in user-scoped `~/.claude.json` C. Both in `~/.claude.json` so no secrets ever enter the repo D. GitHub server in `CLAUDE.md` with the token stored in a `.env` file
- 

**Q27.** The agent burns many turns on exploratory calls just to discover what data exists (which issues, which docs, which tables). What MCP capability addresses this?

- A. Expose content catalogs (issue summaries, documentation hierarchies, database schemas) as **MCP resources** B. Add more tools so each discovery need has a dedicated tool C. Set `tool_choice: "any"` to force faster tool usage D. Increase the context window so exploration is cheaper
- 

## Domain 3 — Claude Code Configuration & Workflows

---

**Q28.** A new team member reports that Claude Code isn't following the team's testing conventions, while existing members have no problem. Where are those conventions most likely defined?

- A. In project-level `.claude/CLAUDE.md` B. In `~/.claude/CLAUDE.md` — user-level configuration, which is not shared via version control C. In `.claude/rules/` with glob patterns D. In `.mcp.json`
- 

**Q29.** You want a `/review` slash command available to every developer automatically when they clone or pull the repository. Where do you create it?

- A. `~/.claude/commands/` B. `.claude/commands/` in the project repository C. In the root `CLAUDE.md` D. In `.claude/config.json` under a `commands` array
- 

**Q30.** Your root `CLAUDE.md` has grown to 900 lines covering testing, API conventions, deployment, and security. What is the most maintainable restructuring?

- A. Delete the least-used sections to shorten it B. Split it into focused topic files in `.claude/rules/`, and/or use `@import` to reference external standards files modularly C. Move everything into a single skill in `.claude/skills/` D. Move it to `~/.claude/CLAUDE.md` so it loads faster
- 

**Q31.** Test files sit next to their source files throughout the codebase (`Button.test.tsx` beside `Button.tsx`), and all tests must follow the same conventions regardless of location. Most maintainable approach?

- A. Consolidate everything in root `CLAUDE.md` under headers, relying on Claude to infer which section applies B. A separate `CLAUDE.md` in every subdirectory containing tests C. Rule files in `.claude/rules/` with YAML frontmatter `paths:` glob patterns such as `**/*.test.tsx` D. Skills in `.claude/skills/` for each code type
- 

**Q32.** Which command tells you which memory files are currently loaded, so you can diagnose inconsistent behaviour across sessions?

- A. `/compact` B. `/memory` C. `/config` D. `--resume`
- 

**Q33.** Your codebase-analysis skill produces hundreds of lines of exploration output that crowds out the main conversation. Which frontmatter option fixes this?

- A. `allowed-tools: [Read, Grep]` B. `argument-hint: "<package>"` C. `context: fork` D. `model: haiku`
- 

**Q34.** You want a skill that can create and modify files but must never run destructive shell commands. Which frontmatter field enforces this?

- A. `context: fork` B. `allowed-tools` C. `argument-hint` D. `paths`
- 

**Q35.** Which statement correctly distinguishes skills from `CLAUDE.md`?

A. Skills are always loaded; CLAUDE.md is invoked on demand B. CLAUDE.md is always loaded and holds universal standards; skills are invoked on demand for task-specific workflows C. Both are always loaded, but skills take precedence D. Skills are user-scoped only; CLAUDE.md is project-scoped only

---

**Q36.** You are restructuring a monolith into microservices — dozens of files, decisions about service boundaries and module dependencies. Which approach?

A. Direct execution with comprehensive upfront instructions detailing each service's structure B. Direct execution incrementally, letting implementation reveal natural service boundaries C. Plan mode, to explore the codebase, understand dependencies, and design the approach before making changes D. Direct execution first, switching to plan mode only if unexpected complexity appears

---

**Q37.** Which task is best suited to **direct execution** rather than plan mode?

A. Migrating a logging library across 45+ files B. Adding a single date-validation conditional to one function with a clear stack trace C. Choosing between two integration approaches with different infrastructure requirements D. Introducing a caching layer that affects several services

---

**Q38.** Your CI job runs `claude "Analyze this pull request for security issues"` and hangs indefinitely; logs show it waiting for interactive input. What is the correct fix?

A. `claude -p "Analyze this pull request for security issues"` B. Set `CLAUDE_HEADLESS=true` before the command C. Redirect stdin: `claude "... " < /dev/null` D. Add the `--batch` flag

---

**Q39.** Your CI review posts findings as inline PR comments. When new commits arrive and the review re-runs, developers complain about duplicate comments on issues they already fixed or discussed. What should you do?

A. Only run the review on the first commit of each PR B. Include the prior review findings in context and instruct Claude to report only new or still-unaddressed issues C. Switch to `--output-format json` so comments post as a single summary D. Increase the model's temperature so it varies its findings

---

## Domain 4 — Prompt Engineering & Structured Output

---

**Q40.** Your CI review flags too many false positives on comment accuracy. Your current prompt says "check that comments are accurate" and "only report high-confidence findings." What most improves precision?



A. Add "be conservative" and "avoid speculation" to the prompt B. Replace the vague instruction with explicit criteria, e.g. "flag a comment only when the claimed behaviour contradicts the actual code behaviour" C. Lower the model temperature to 0 D. Ask the model to output a confidence score and filter below 0.8

---

**Q41.** One review category (style suggestions) produces so many false positives that developers have started ignoring all review output, including accurate security findings. What is a legitimate short-term action?

A. Temporarily disable the high false-positive category to restore trust while you improve its prompt B. Reduce the total number of findings reported per PR to five C. Post all findings to a separate channel developers can ignore D. Require two independent runs to agree before posting anything

---

**Q42.** Detailed written instructions still produce inconsistently formatted review output — sometimes with a suggested fix, sometimes not; sometimes with a line number, sometimes not. What technique best fixes this?

A. Increase `max_tokens` B. Add 2–4 few-shot examples demonstrating the exact desired output format (location, issue, severity, suggested fix) C. Move the formatting instructions to the end of the prompt D. Use `tool_choice: "auto"`

---

**Q43.** You add few-shot examples for ambiguous tool-selection cases. What should each example include to maximise generalisation to novel patterns?

A. Only the input and the chosen tool name, kept minimal to save tokens B. The reasoning for why that action was chosen over plausible alternatives C. The full JSON schema of the chosen tool D. A confidence score for the selection

---

**Q44.** What is the most reliable approach for guaranteed schema-compliant structured output from Claude?

A. Asking in the prompt for JSON only, then parsing with a permissive parser B. Tool use ( `tool_use` ) with a JSON schema as the tool's input schema C. Post-processing free-text output with a regex extractor D. Requesting YAML, which is more forgiving than JSON

---

**Q45.** After moving to `tool_use` with a strict JSON schema, you still see invoices where the line items don't add up to the stated total. Why?

A. The schema needs `additionalProperties: false` B. Strict schemas eliminate syntax errors but do not prevent semantic errors; correctness of values requires separate validation logic C. The model's temperature is too high D. The schema fields should all be marked required

---

**Q45b — (numbered Q46 below).**

**Q46.** Extraction results show fabricated vendor names for documents that genuinely have no vendor listed. Your schema marks `vendor_name` as required. What is the fix?

- A. Add a few-shot example showing an invented placeholder value
  - B. Make `vendor_name` optional/nullable so the model can honestly return null rather than fabricating a value to satisfy the schema
  - C. Lower `max_tokens` to discourage elaboration
  - D. Add `"unknown"` as the default value in the schema
- 

**Q47.** Your document type classifier enum has five values, but new document types keep appearing that don't fit. Which schema pattern handles extensibility and ambiguity?

- A. Remove the enum and accept any free-text string
  - B. Add an `"other"` enum value with a companion detail string field, and an `"unclear"` value for ambiguous cases
  - C. Add all possible future types to the enum upfront
  - D. Split the field into five booleans
- 

**Q48.** Schema validation fails because a date was formatted `03/04/2026` instead of ISO-8601. What retry strategy is most effective?

- A. Resend the identical request; the model may format it correctly by chance
  - B. Send a follow-up including the original document, the failed extraction, and the specific validation error for self-correction
  - C. Escalate the document to human review immediately
  - D. Strip the date field from the schema
- 

**Q49.** Extraction repeatedly fails to populate `contract_end_date` for a batch of documents. Investigation shows the end date is only stated in a separate appendix that was never provided to the model. What should you conclude?

- A. Retry with a stronger prompt emphasising the field's importance
  - B. Retry will be ineffective — the information is absent from the source provided; retries fix format and structural errors, not missing information
  - C. Add the field to `required` so the model must find it
  - D. Add few-shot examples showing end dates being extracted
- 

**Q50.** Your manager proposes moving both workflows to the Message Batches API for 50% savings: (1) a blocking pre-merge check developers wait on, and (2) an overnight technical debt report. How should you respond?

- A. Move both, polling for completion
  - B. Move the overnight technical debt report to batch; keep the pre-merge check on the synchronous API
  - C. Keep both synchronous, since batch results cannot be correlated
  - D. Move both, with a timeout fallback to real-time
- 

**Q51.** A 14-file pull request review produces inconsistent results: detailed feedback on some files, superficial on others, obvious bugs missed, and a pattern flagged in one file but approved in another. Best restructuring?

A. Switch to a model with a larger context window so all 14 files get adequate attention in one pass B. Split into focused passes: per-file analysis for local issues, plus a separate integration pass for cross-file data flow C. Require developers to submit PRs of no more than 3–4 files D. Run three independent passes and only flag issues appearing in at least two

---

## Domain 5 — Context Management & Reliability

---

**Q52.** After a long support conversation with progressive summarisation, the agent issues a refund of the wrong amount. The summary read "customer wanted a refund for the damaged item" — the original message specified \$249.99 and a full refund rather than store credit. What is the fix?

A. Summarise more aggressively to leave room for detail B. Maintain a persistent "case facts" block (amounts, dates, order numbers, customer-stated expectations) included in every prompt, outside the summarised history C. Increase the summarisation frequency D. Ask the customer to restate the amount each turn

---

**Q53.** A synthesis agent receives ten subagent reports concatenated into one long input and reliably reflects the first two and last two, omitting findings from reports 4–7. What is happening and how do you mitigate it?

A. Token truncation; increase `max_tokens` B. The "lost in the middle" effect; place a key findings summary at the beginning and organise details with explicit section headers C. Tool result caching; disable prompt caching D. Rate limiting; add retries between subagent calls

---

**Q54.** Each `lookup_order` call returns 40+ fields, but only about 5 are relevant to a return request. Context fills rapidly across a multi-issue conversation. Best approach?

A. Increase the context window by switching model tiers B. Trim verbose tool outputs to only the relevant fields before they accumulate in context C. Summarise the whole conversation after every tool call D. Move the order data into the system prompt

---

**Q55.** During a long codebase exploration session the agent begins giving inconsistent answers and referring to "typical patterns" rather than the specific classes it identified earlier. Which combination best addresses this? (*Select the single best answer.*)

A. Restart the session from scratch each time this happens B. Have the agent maintain scratchpad files recording key findings and reference them for later questions, delegate verbose exploration to subagents, and use `/compact` when context fills with discovery output C. Increase `max_tokens` and lower temperature D. Load all source files into the initial prompt so nothing is forgotten

---

**Q56.** A customer opens with: "I don't want to deal with a bot, put me through to a human." The issue itself looks like a routine damage replacement the agent could resolve. What should the agent do?

- A. Attempt resolution first, since first-contact resolution is the target metric, and escalate only if it fails
  - B. Honour the explicit request and escalate to a human immediately
  - C. Run sentiment analysis to confirm frustration before escalating
  - D. Ask three clarifying questions to determine whether escalation is warranted
- 

**Q57.** Your agent escalates 45% of cases, far above target. It escalates straightforward cases with clear evidence while attempting complex policy-exception cases autonomously. What is the most effective **first** intervention?

- A. Have the agent self-report a 1–10 confidence score and route below a threshold to humans
  - B. Add explicit escalation criteria to the system prompt with few-shot examples showing when to escalate versus resolve autonomously
  - C. Train a separate classifier on historical tickets to predict escalation need
  - D. Add sentiment analysis and escalate when negative sentiment exceeds a threshold
- 

**Q58.** A subagent's search times out. Which error propagation design best enables intelligent coordinator recovery?

- A. Catch the timeout and return an empty result set marked successful
  - B. Retry with exponential backoff internally, then return a generic "search unavailable" status
  - C. Return structured error context: failure type, the attempted query, any partial results, and potential alternative approaches
  - D. Propagate the exception to a top-level handler that terminates the research workflow
- 

**Q59.** Two credible sources report different figures for the same statistic (34% vs 41%). How should the synthesis output handle this?

- A. Select the value from the more recent source and discard the other
  - B. Average the two values
  - C. Preserve both values with source attribution, annotate the conflict, and structure the report to distinguish well-established from contested findings
  - D. Omit the statistic entirely to avoid publishing uncertainty
- 

**Q60.** Your extraction pipeline reports 97% overall accuracy, and leadership wants to remove human review for high-confidence extractions. What should you do before agreeing?

- A. Accept — 97% exceeds the threshold for automation
  - B. Analyse accuracy by document type and by field to confirm consistent performance across all segments, and implement stratified random sampling of high-confidence extractions to measure ongoing error rates and detect novel error patterns
  - C. Raise the confidence threshold to 0.99 and automate everything above it
  - D. Ask the model to self-report whether each extraction should be reviewed
- 
-

## PART 2 — ANSWER KEY WITH JUSTIFICATIONS

---

### Q1 — B

`stop_reason` is the designed control signal for the agentic loop:

`"tool_use"` means continue, `"end_turn"` means stop.

- **A** is a symptom-level proxy — it happens to correlate but isn't the documented mechanism, and breaks in mixed content cases.
- **C** is an anti-pattern as a *primary* stopping mechanism (fine as a safety cap, but it doesn't tell you whether work is done).
- **D** is explicitly listed as an anti-pattern: parsing natural-language signals for termination is unreliable.

### Q2 — A

Tool results must be **appended to conversation history** so the model can incorporate the new information into its reasoning on the next iteration. That accumulation *is* the agent's working memory.

- **B** is unrelated infrastructure. **C** destroys the record of what Claude requested. **D** misuses the system prompt and breaks the `tool_use/tool_result` pairing.

### Q3 — C

The defining distinction is **model-driven decision-making** — Claude reasons about the next action from current context — versus pre-configured decision trees or fixed tool sequences.

- **A, B, D** describe incidental implementation details, not the architectural distinction.

### Q4 — B

The `Task` tool is the mechanism for spawning subagents, and the coordinator's `allowedTools` **must include** `"Task"` or delegation is impossible. This is the first thing to check.

- **A** matters later but doesn't stop spawning. **C** is for branching sessions, not spawning. **D** has no effect on capability.

### Q5 — B

Parallelism comes from the coordinator emitting **multiple `Task` tool calls in a single response**. Across separate turns they execute sequentially.

- **A** forces a tool call but doesn't parallelise. **C** isolates context, not scheduling. **D** doesn't change turn structure.

### Q6 — B

Subagents operate with **isolated context** and do **not** inherit the coordinator's conversation history. Prior findings must be placed **directly in the subagent's prompt**.

- **A** blames capacity for a context-passing bug. **C** would show as empty input, not contradiction. **D** confuses spawning permission with context.

#### Q7 — C

The coordinator's own logs are the evidence: it decomposed "creative industries" into three *visual-arts* subtasks, omitting music, writing and film. The subagents executed correctly — the assignments were wrong.

- **A, B, D** blame downstream agents that performed correctly within the scope they were given. **Blame the assignment, not the worker.**

#### Q8 — C

When a tool sequence is required for critical business logic (identity before financial action), **programmatically enforcement gives deterministic guarantees**; prompts do not.

- **A** and **B** rely on probabilistic LLM compliance — unacceptable at 12% failure with financial consequences.
- **D** addresses *tool availability*, but the actual defect is *tool ordering*.

#### Q9 — C

A tool-call interception hook blocks the policy-violating call before it executes and redirects to escalation — a **guaranteed** outcome.

- **A** and **B** are probabilistic. **D** detects the violation only *after* the refund has already gone out.

#### Q10 — B

**PostToolUse** hooks intercept tool **results** for transformation before the model processes them — exactly the right place for normalising heterogeneous formats.

- **A** and **C** make normalisation the model's probabilistic job. **D** invents an awkward workflow and still burns turns.

#### Q11 — C

The prescribed pattern is: decompose multi-concern requests into distinct items → investigate each **in parallel using shared context** → synthesise a **unified** resolution.

- **A** and **D** damage the customer experience and hurt first-contact resolution. **B** escalates something well within agent capability.

#### Q12 — B

Because the human **lacks the transcript**, the handoff must be a structured summary: customer ID, root cause, refund amount, recommended action.

- **A** shifts work to the human and delays resolution. **C** dumps unprocessed data with no analysis. **D** provides metrics, not actionable case context.

#### Q13 — B

Open-ended tasks with unknown structure call for **dynamic adaptive decomposition**: map structure, find high-impact areas, build a prioritised plan that adapts as dependencies emerge.

- **A** imposes a fixed order on an unknown problem. **C** blows context and dilutes attention. **D** ignores that batch can't do multi-turn tool calling and can't adapt.

#### Q14 — B

**Prompt chaining** — a fixed sequential pipeline of focused passes — suits predictable multi-aspect reviews, and the per-file + cross-file split prevents attention dilution.

- **A** is for open-ended investigation. **C** causes exactly the inconsistency the pattern avoids. **D** produces uncoordinated, duplicative findings.

#### Q15 — C

`fork_session` creates **independent branches from a shared analysis baseline** — precisely for exploring divergent approaches without repeating the analysis.

- **A** continues one linear session. **B** compresses context. **D** isolates a *skill's* execution context, a different concept.

#### Q16 — C

When prior **tool results are stale**, starting a new session with a **structured summary** is more reliable than resuming into out-of-date data.

- **A** assumes the model will notice staleness. **B** is irrelevant. **D** compacts context but doesn't correct stale facts already in it.

#### Q17 — B

**Tool descriptions are the primary mechanism LLMs use for tool selection.** Minimal descriptions are the root cause; expanding them with input formats, example queries, edge cases and boundaries is the low-effort, high-leverage fix — and the question asks for the **first step**.

- **A** adds token overhead without fixing the cause. **C** is over-engineered and bypasses the model's language understanding. **D** is a legitimate architecture change but far more effort than a first step warrants.

#### Q18 — B

Renaming to `extract_web_results` with a web-specific description **eliminates the functional overlap** that causes misrouting.

- **A** loses a capability. **C** leaves the descriptions ambiguous, so the model still can't choose. **D** forces a tool call, not the *right* one.

#### Q19 — C

System prompt wording matters: **keyword-sensitive instructions can create unintended tool associations** that override well-written descriptions.

- **A, B, D** have no bearing on selection reasoning.

#### Q20 — B

Structured error metadata ( `errorCategory` , `isRetryable` , human-readable description) lets the agent make **appropriate recovery decisions**. Uniform generic errors prevent this entirely.

- **A** is unstructured and inconsistently interpreted. **C** is insufficient categorisation. **D** silently disguises failure as success — an explicit anti-pattern.

#### Q21 — B

A policy violation is a **business error**: non-retryable, and it needs a customer-friendly explanation so the agent can communicate rather than retry.

- **A** would cause pointless retries. **C** suppresses the error. **D** mislabels a policy outcome as an authorisation problem.

#### Q22 — B

**Access failures** (timeouts needing a retry/alternative decision) and **valid empty results** (a successful query with no matches) are semantically different; conflating them produces wrong recovery behaviour.

- **A** is irrelevant. **C** over-escalates normal results. **D** would retry queries that legitimately have no answer.

#### Q23 — B

Too many tools (18 vs 4–5) **degrades selection reliability** by increasing decision complexity, and agents with out-of-specialisation tools misuse them. The fix is scoped tool access with narrow cross-role additions only for high-frequency needs.

- **A, C, D** address unrelated concerns.

#### Q24 — B

A **scoped `verify_fact` tool** applies least privilege: it serves the 85% common case directly while preserving coordination for the 15% complex cases.

- **A** over-provisions and violates separation of concerns. **B...** (see A). **C** creates blocking dependencies since later synthesis steps may depend on earlier verified facts. **D** relies on speculative caching that can't predict what will need verifying.



## Q25 — C

**Forced tool selection** — `tool_choice: {"type": "tool", "name": "extract_metadata"}` — guarantees that specific tool is called; subsequent steps proceed in follow-up turns.

- **A** is probabilistic. **B** forces a tool, not *that* tool. **D** array order carries no ordering guarantee.

## Q26 — B

Project-scoped `.mcp.json` shares team tooling via version control, with **environment variable expansion** (`${GITHUB_TOKEN}`) keeping the secret out of the repo; personal/experimental servers belong in user-scoped `~/.claude.json`. Both are available simultaneously.

- **A** risks committing a secret and doesn't share. **C** means the team doesn't get the shared server. **D** misuses `CLAUDE.md`, which is not an MCP configuration file.

## Q27 — A

**MCP resources** exist to expose content catalogs — issue summaries, documentation hierarchies, database schemas — giving the agent visibility into available data **without exploratory tool calls**. (Tools = actions; resources = content.)

- **B** worsens tool-count-induced selection degradation. **C** forces calls without informing them. **D** makes waste cheaper rather than removing it.

## Q28 — B

User-level `~/.claude/CLAUDE.md` applies only to that user and is **not shared via version control** — which is exactly why an existing member has it and a new one doesn't. Move the conventions to project level.

- **A** and **C** would have propagated automatically on clone. **D** configures MCP servers, not conventions.

## Q29 — B

**Project-scoped slash commands live in** `.claude/commands/` in the repository — version-controlled and automatically available to everyone on clone or pull.

- **A** is personal and not shared. **C** is for project instructions and context, not command definitions. **D** describes a mechanism that does not exist in Claude Code.

## Q30 — B

`.claude/rules/` holds focused topic-specific rule files as an alternative to a monolithic `CLAUDE.md`, and `@import` keeps `CLAUDE.md` modular by referencing external standards files.

- **A** loses information. **C** makes always-on standards on-demand, changing their semantics. **D** would stop sharing them with the team

entirely.

#### Q31 — C

`.claude/rules/` files with YAML frontmatter **glob patterns** apply conventions by file type regardless of directory — essential when test files are scattered — and load only when editing matching files, reducing irrelevant context.

- **A** relies on inference rather than explicit matching. **B** is directory-bound and unmaintainable across many directories. **D** requires manual invocation, contradicting the need for automatic path-based application.

#### Q32 — B

`/memory` verifies which memory files are loaded and is the documented way to diagnose inconsistent cross-session behaviour.

- **A** compresses context. **C** is not the memory inspection command. **D** resumes a session.

#### Q33 — C

`context: fork` runs the skill in an **isolated sub-agent context**, so verbose output never pollutes the main conversation.

- **A** restricts tools, not context. **B** prompts for arguments. **D** is not the mechanism for this problem.

#### Q34 — B

`allowed-tools` in SKILL.md frontmatter **restricts tool access during skill execution** — e.g. limiting to file write operations to prevent destructive actions.

- **A** isolates context. **C** prompts for parameters. **D** belongs to `.claude/rules/` path scoping.

#### Q35 — B

**CLAUDE.md = always loaded, universal standards. Skills = on-demand invocation for task-specific workflows.** That is the documented decision rule.

- **A** inverts it. **C** and **D** are factually wrong; both support project and user scope.

#### Q36 — C

Plan mode is designed for exactly this profile: large-scale change, multiple valid approaches, architectural decisions, multi-file modification. It enables safe exploration and design before committing to changes, preventing costly rework.

- **A** assumes you already know the right structure without exploring. **B** risks discovering dependencies too late. **D** ignores that the complexity is stated upfront, not emergent.

### Q37 — B

Direct execution suits **simple, well-scoped changes** — a single-file fix with a clear stack trace and one conditional to add.

- **A, C, D** all involve multi-file scope, architectural choice, or infrastructure implications → plan mode.

### Q38 — A

The `-p / --print` flag runs Claude Code non-interactively: it processes the prompt, writes to stdout, and exits — exactly what CI requires.

- **B** and **D** reference non-existent features. **C** is a Unix workaround that doesn't address the command's interactive mode properly.

### Q39 — B

Including **prior review findings in context** and instructing Claude to report only **new or still-unaddressed** issues is the documented approach to avoiding duplicate PR comments.

- **A** abandons review value on later commits. **C** changes output format, not duplication. **D** introduces randomness, not relevance.

### Q40 — B

**Explicit categorical criteria outperform vague quality**

**instructions.** "Flag only when claimed behaviour contradicts actual behaviour" is a testable boundary; "be accurate" and "high confidence" are not.

- **A** adds more vague language — explicitly noted as ineffective. **C** doesn't change judgement criteria. **D** relies on poorly calibrated self-reported confidence for filtering.

### Q41 — A

High false-positive categories undermine trust in the accurate categories too. **Temporarily disabling the noisy category** restores developer trust while the prompt for it is improved — a documented, legitimate tactic.

- **B** arbitrarily drops real findings. **C** removes visibility without fixing the cause. **D** suppresses genuine intermittently-detected bugs.

### Q42 — B

**Few-shot examples are the most effective technique** for consistently formatted, actionable output when detailed instructions alone produce inconsistency — including the exact field set (location, issue, severity, suggested fix).

- **A** addresses length, not structure. **C** is a marginal tweak. **D** is unrelated to formatting.

### Q43 — B

Examples should **show the reasoning for choosing one action over plausible alternatives**. That reasoning is what enables **generalisation to novel patterns** rather than matching only the pre-specified cases.

- **A** removes exactly the signal that generalises. **C** is already available in the tool definition. **D** adds unreliable self-assessment.

#### Q44 — B

**Tool use with a JSON schema** is the most reliable path to guaranteed schema-compliant output and **eliminates JSON syntax errors**.

- **A, C, D** all rely on post-hoc parsing of free-form generation and provide no compliance guarantee.

#### Q45 — B

Strict schemas guarantee **shape**, not **meaning**. Line items not summing to the total is a **semantic** error a schema will happily accept. Catch it with validation logic — e.g. extracting `calculated_total` alongside `stated_total` and flagging discrepancies.

- **A** restricts extra keys only. **C** doesn't address arithmetic correctness. **D** would make things worse by forcing fabrication.

#### Q46 — B

Design fields as **optional/nullable when source documents may not contain them**. Required fields pressure the model into **fabricating values to satisfy the schema**.

- **A** teaches fabrication explicitly. **C** is unrelated. **D** substitutes a fake certainty ("unknown" as a *value*) for an honest null, and still doesn't remove the required constraint.

#### Q47 — B

The documented pattern: an `"other"` **enum value plus a detail string** for extensible categorisation, and an `"unclear"` value so ambiguous cases have an honest option.

- **A** loses all validation. **C** is impossible to maintain and still fails on the next new type. **D** allows contradictory multi-true states.

#### Q48 — B

**Retry with error feedback**: send the original document, the failed extraction, and the **specific validation errors** so the model can self-correct. Format mismatches are precisely the class of error retries fix.

- **A** relies on chance. **C** wastes reviewer capacity on a mechanically fixable issue. **D** discards required data.

#### Q49 — B

Retries are **ineffective when the required information is simply absent from the source**, as opposed to format or structural errors. Recognising this prevents burning cost on doomed retries — the real fix is supplying the appendix.

- **A** and **D** cannot conjure absent data. **C** actively increases hallucination risk.

#### Q50 — B

The Message Batches API gives **50% savings** but has **up to a 24-hour window with no guaranteed latency SLA** — unsuitable for a blocking pre-merge check where developers wait, ideal for an overnight report.

- **A** and **D** gamble on batches being "usually faster," unacceptable for blocking workflows (and **D** adds needless complexity). **C** rests on a misconception: batch results are correlatable via `custom_id`.

#### Q51 — B

The symptoms — uneven depth, missed bugs, contradictory findings across files — are classic **attention dilution**. Per-file passes give consistent depth; a separate integration pass catches cross-file issues.

- **A** misunderstands that a larger context window does not fix attention quality. **C** pushes the burden onto developers without improving the system. **D** suppresses real bugs that are detected intermittently.

#### Q52 — B

Progressive summarisation condenses exactly the wrong things: **numerical values, dates, and customer-stated expectations**. Extracting them into a persistent **"case facts" block outside the summarised history** preserves them verbatim across the whole conversation.

- **A** and **C** accelerate the information loss. **D** pushes the system's failure onto the customer.

#### Q53 — B

This is the **"lost in the middle"** effect: models process the beginning and end of long inputs reliably and may omit middle sections. Mitigate by putting a **key findings summary at the beginning** and using **explicit section headers**.

- **A** confuses output length with input attention. **C** and **D** describe unrelated mechanisms.

#### Q54 — B

Tool results accumulate in context and consume tokens **disproportionately to their relevance**. Trimming outputs to the relevant fields *before* they accumulate is the documented fix.

- **A** raises the ceiling without fixing the waste. **C** triggers the

summarisation-loss problem. **D** misuses the system prompt for per-call data.

#### Q55 — B

Inconsistent answers plus references to "typical patterns" instead of specific discovered classes is the signature of **context degradation** in extended sessions. The documented toolkit is **scratchpad files** for persistence, **subagent delegation** to isolate verbose exploration, and **/compact** to reduce context usage.

- **A** discards all accumulated understanding. **C** doesn't address context. **D** guarantees exhaustion — the explicit anti-pattern is reading everything upfront.

#### Q56 — B

When a customer **explicitly requests a human**, honour it **immediately** without first attempting investigation. (Contrast: mere *frustration* with a solvable issue → acknowledge and offer to resolve, escalating only if they reiterate.)

- **A** ignores an explicit preference to chase a metric. **C** uses sentiment, which does not correlate with case complexity. **D** adds friction after a clear request.

#### Q57 — B

The root cause is **unclear decision boundaries**, and explicit escalation criteria with few-shot examples is the proportionate first response before adding infrastructure.

- **A** fails because LLM self-reported confidence is poorly calibrated — the agent is already wrongly confident on the hard cases. **C** is over-engineered before prompt optimisation has been tried. **D** solves a different problem: sentiment ≠ complexity.

#### Q58 — C

**Structured error context** — failure type, attempted query, partial results, potential alternatives — gives the coordinator what it needs to retry with a modified query, try an alternative, or proceed with partial results and annotate coverage gaps.

- **A** silently marks failure as success, preventing recovery and risking incomplete output. **B**'s generic status hides the context the coordinator needs. **D** terminates the whole workflow when recovery could have succeeded.

#### Q59 — C

Conflicting statistics from credible sources should be **annotated with source attribution for both values**, letting the coordinator decide reconciliation, with the report structured to distinguish **well-established from contested** findings and preserve methodological context.

- **A** arbitrarily discards evidence (and recency isn't automatically

correctness — this is also why publication dates must be captured).  
**B** invents a number that appears in no source. **D** hides material information.

**Q60 — B**

An aggregate figure like **97% can mask poor performance on a specific document type or field**. Validate accuracy **by document type and field segment** before reducing review, and use **stratified random sampling** of high-confidence extractions for ongoing error measurement and **novel error pattern detection**.

- **A** acts on the masked aggregate. **C** raises a threshold whose calibration hasn't been validated. **D** relies on unvalidated model self-assessment.

## SCORE YOURSELF

| Score | Reading                                                        |
|-------|----------------------------------------------------------------|
| 52-60 | Comfortably exam-ready                                         |
| 43-51 | Around the pass line — review every miss and its justification |
| 34-42 | Not yet — rework the weakest domain from the Study Guide       |
| < 34  | Restart with the Study Guide before attempting more mocks      |

**Domain breakdown for targeted revision:**

- Q1-Q16 → Domain 1 (Agentic Architecture & Orchestration)
- Q17-Q27 → Domain 2 (Tool Design & MCP Integration)
- Q28-Q39 → Domain 3 (Claude Code Configuration & Workflows)
- Q40-Q51 → Domain 4 (Prompt Engineering & Structured Output)
- Q52-Q60 → Domain 5 (Context Management & Reliability)

If you drop more than 3 in any single domain, re-read that section of the Study Guide before your next attempt.

*These are original practice questions written against the publicly published official Exam Guide (v1.0, July 2026) objectives. They are not actual exam items — real exam content is confidential under the NDA you accept before testing.*